

Лекция 6: Строки (String)

Введение

Строки — один из самых часто используемых типов данных в программировании. В Java строка представляет собой последовательность символов и реализована как класс `String`. В отличие от многих других языков, где строка — это просто массив символов, в Java `String` — это полноценный объект со своими методами.

Важнейшая особенность строк в Java: **они неизменяемы (immutable)**. Это значит, что после создания строки её нельзя изменить. Любая операция, которая кажется изменяющей строку (например, конкатенация или замена символов), на самом деле создаёт **новую** строку.

Сегодня мы изучим:

- Создание строк и основные методы
- Сравнение строк (правильный способ!)
- Поиск и извлечение подстрок
- Преобразование строк
- Классы `StringBuilder` и `StringBuffer` для эффективной работы с изменяемыми строками

1) Основы работы со строками

Создание строк

```
public class StringCreation {  
    public static void main(String[] args) {  
        // Простейший способ: строковый литерал  
        String str1 = "Hello, World!";  
  
        // Через конструктор (реже, но бывает нужно)  
        String str2 = new String("Hello, World!");  
  
        // Из массива символов  
        char[] chars = {'J', 'a', 'v', 'a'};  
        String str3 = new String(chars); // "Java"  
  
        // Пустая строка
```

```
String empty = "";

// null — отсутствие строки (не пустая строка!)
String nullString = null;
}
}
```

Важно: строковые литералы (заклЮчЁнные в двойные кавычки) хранятся в специальном пуле строк (String Pool), что позволяет экономить память. Строки, созданные через new, всегда создают новый объект.

Длина строки (метод `length()`)

```
public class StringLength {
    public static void main(String[] args) {
        String str = "Java Programming";
        int len = str.length(); // 16 (пробел тоже считается)
        System.out.println("Длина строки: " + len);

        String empty = "";
        System.out.println("Длина пустой строки: " + empty.length()); // 0

        // Осторожно с null!
        String nullStr = null;
        // System.out.println(nullStr.length()); // NullPointerException!
    }
}
```

Доступ к символу (метод `charAt()`)

```
public class StringCharAt {
    public static void main(String[] args) {
        String str = "Hello";

        for (int i = 0; i < str.length(); i++) {
            System.out.println("Символ на позиции " + i + ": " + str.charAt(i));
        }
        // Вывод: H, e, l, l, o

        // charAt возвращает char, можно сравнивать
        if (str.charAt(0) == 'H') {
            System.out.println("Первый символ - H");
        }
    }
}
```

```
}  
}
```

Конкатенация (объединение строк)

```
public class StringConcat {  
    public static void main(String[] args) {  
        // Оператор + работает для строк  
        String firstName = "Иван";  
        String lastName = "Петров";  
        String fullName = firstName + " " + lastName;  
        System.out.println(fullName); // "Иван Петров"  
  
        // Конкатенация с числами  
        int age = 25;  
        String message = "Мне " + age + " лет";  
        System.out.println(message); // "Мне 25 лет"  
  
        // Важно: порядок имеет значение  
        System.out.println(10 + 20 + " = сумма"); // "30 = сумма"  
        System.out.println("Сумма = " + 10 + 20); // "Сумма = 1020"  
  
        // Метод concat()  
        String s1 = "Hello";  
        String s2 = s1.concat(" World");  
        System.out.println(s2); // "Hello World"  
    }  
}
```

2) Сравнение строк

Это одна из самых частых ошибок у начинающих.

Оператор `==` сравнивает **ссылки** (адреса в памяти), а не содержимое строк.
Для сравнения содержимого используется метод `equals()`.

Правильное и неправильное сравнение

```
public class StringComparison {  
    public static void main(String[] args) {  
        String s1 = "Hello";  
        String s2 = "Hello";  
        String s3 = new String("Hello");
```

```

// Сравнение через == (сравнивает ссылки!)
System.out.println("s1 == s2: " + (s1 == s2)); // true (оба из пула)
System.out.println("s1 == s3: " + (s1 == s3)); // false (разные объекты)

// Правильное сравнение через equals
System.out.println("s1.equals(s2): " + s1.equals(s2)); // true
System.out.println("s1.equals(s3): " + s1.equals(s3)); // true

// Сравнение без учёта регистра
String t1 = "Java";
String t2 = "java";
System.out.println("t1.equals(t2): " + t1.equals(t2)); // false
System.out.println("t1.equalsIgnoreCase(t2): " + t1.equalsIgnoreCase(t2)); // true
}
}

```

Золотое правило: Всегда используйте `equals()` для сравнения содержимого строк, никогда `==` (кроме редких случаев, когда вы точно знаете, что делаете).

Пустые строки и null

```

public class EmptyVsNull {
    public static void main(String[] args) {
        String empty = ""; // пустая строка (0 символов)
        String nullStr = null; // отсутствие строки

        // Проверка на пустоту
        System.out.println(empty.isEmpty()); // true
        // System.out.println(nullStr.isEmpty()); // NullPointerException!

        // Безопасная проверка
        if (nullStr != null && !nullStr.isEmpty()) {
            System.out.println("Строка не пуста");
        }

        // Современный метод (Java 11+)
        System.out.println(empty.isBlank()); // true (учитывает пробелы)
        System.out.println(" ".isBlank()); // true
    }
}

```

3) Поиск и извлечение подстрок

Поиск символа или подстроки

```
public class StringSearch {
    public static void main(String[] args) {
        String text = "Java programming is fun. Java is powerful.";

        // indexOf() — первое вхождение
        System.out.println("indexOf('a'): " + text.indexOf('a')); // 1
        System.out.println("indexOf('Java'): " + text.indexOf("Java")); // 0
        System.out.println("indexOf('Python'): " + text.indexOf("Python")); // -1 (не найдено)

        // indexOf с начальной позиции
        System.out.println("indexOf('a', 2): " + text.indexOf('a', 2)); // 3

        // lastIndexOf() — последнее вхождение
        System.out.println("lastIndexOf('a'): " + text.lastIndexOf('a')); // 37
        System.out.println("lastIndexOf('Java'): " + text.lastIndexOf("Java")); // 23

        // contains() — проверка наличия
        if (text.contains("fun")) {
            System.out.println("Слово 'fun' найдено");
        }

        // startsWith() / endsWith()
        System.out.println("startsWith('Java'): " + text.startsWith("Java")); // true
        System.out.println("endsWith('ful.'): " + text.endsWith("ful.")); // true
    }
}
```

Извлечение подстроки (substring)

```
public class StringSubstring {
    public static void main(String[] args) {
        String str = "Hello, World!";

        // substring(startIndex) — от startIndex до конца
        String sub1 = str.substring(7);
        System.out.println(sub1); // "World!"
    }
}
```

*// substring(startIndex, endIndex) — от startIndex включительно до endIndex
исключительно*

```
String sub2 = str.substring(7, 12);  
System.out.println(sub2); // "World"
```

// Пример: получение расширения файла

```
String filename = "document.pdf";  
int dotIndex = filename.lastIndexOf('.');  
if (dotIndex != -1) {  
    String extension = filename.substring(dotIndex + 1);  
    System.out.println("Расширение: " + extension); // "pdf"  
}  
}
```

Разделение строки (split)

```
import java.util.Arrays;
```

```
public class StringSplit {  
    public static void main(String[] args) {  
        String text = "яблоко,банан,апельсин,груша";  
  
        // Разделение по запятой  
        String[] fruits = text.split(",");  
        System.out.println("Массив фруктов: " + Arrays.toString(fruits));  
  
        // Разделение по пробелам  
        String sentence = "Java is awesome";  
        String[] words = sentence.split(" ");  
        for (String word : words) {  
            System.out.println("Слово: " + word);  
        }  
  
        // Разделение с ограничением количества частей  
        String data = "один,два,три,четыре,пять";  
        String[] parts = data.split(",", 3);  
        System.out.println(Arrays.toString(parts)); // [один, два, три,четыре,пять]  
    }  
}
```

4) Изменение регистра и замена

toLowerCase() и toUpperCase()

```
public class StringCase {
    public static void main(String[] args) {
        String str = "Java Programming";

        System.out.println(str.toLowerCase()); // "java programming"
        System.out.println(str.toUpperCase()); // "JAVA PROGRAMMING"

        // Исходная строка не изменилась (неизменяемость!)
        System.out.println(str); // "Java Programming"

        // Полезно для сравнения без учёта регистра
        String userInput = "Yes";
        if (userInput.toLowerCase().equals("yes")) {
            System.out.println("Пользователь согласился");
        }
    }
}
```

Замена символов и подстрок

```
java
public class StringReplace {
    public static void main(String[] args) {
        String str = "Hello, World!";

        // replace(char oldChar, char newChar)
        String replaced1 = str.replace('o', '0');
        System.out.println(replaced1); // "Hell0, W0rld!"

        // replace(CharSequence target, CharSequence replacement)
        String replaced2 = str.replace("World", "Java");
        System.out.println(replaced2); // "Hello, Java!"

        // replaceAll() — с использованием регулярных выражений
        String text = "My phone: 123-456-7890";
        String digitsOnly = text.replaceAll("[^0-9]", "");
        System.out.println(digitsOnly); // "1234567890"

        // replaceFirst() — только первое вхождение
        String sentence = "one two one two";
        String replaced3 = sentence.replaceFirst("one", "three");
    }
}
```

```

        System.out.println(replaced3); // "three two one two"
    }
}

```

5) Преобразование строк

Из других типов в строку

```

public class ToString {
    public static void main(String[] args) {
        // Из числа
        int num = 123;
        String str1 = String.valueOf(num);    // "123"
        String str2 = Integer.toString(num);  // "123"
        String str3 = "" + num;               // "123" (неэффективно, но работает)

        // Из double
        double pi = 3.14159;
        String str4 = String.valueOf(pi);     // "3.14159"

        // Из char
        char ch = 'A';
        String str5 = String.valueOf(ch);     // "A"

        // Из boolean
        boolean flag = true;
        String str6 = String.valueOf(flag);   // "true"
    }
}

```

Из строки в другие типы

```

public class ParseString {
    public static void main(String[] args) {
        // B int
        String numStr = "123";
        int num = Integer.parseInt(numStr);   // 123

        // B double
        String piStr = "3.14";
        double pi = Double.parseDouble(piStr); // 3.14
    }
}

```



```

// В boolean (true, если строка (без учёта регистра) равна "true")
boolean b1 = Boolean.parseBoolean("true"); // true
boolean b2 = Boolean.parseBoolean("True"); // true
boolean b3 = Boolean.parseBoolean("false"); // false
boolean b4 = Boolean.parseBoolean("hello"); // false

// В char — нет прямого метода, можно взять первый символ
String s = "A";
char c = s.charAt(0); // 'A'

// Обработка ошибок (NumberFormatException)
String invalid = "123abc";
try {
    int bad = Integer.parseInt(invalid);
} catch (NumberFormatException e) {
    System.out.println("Ошибка: не число!");
}
}
}

```

Преобразование в массив символов

```

public class StringToArray {
    public static void main(String[] args) {
        String str = "Java";

        // В массив char
        char[] chars = str.toCharArray();
        for (char c : chars) {
            System.out.print(c + " "); // J a v a
        }

        // Обратно из массива
        char[] letters = {'H', 'e', 'l', 'l', 'o'};
        String newStr = new String(letters);
        System.out.println("\n" + newStr); // "Hello"
    }
}

```

6) Классы StringBuilder и StringBuffer

Так как `String` неизменяем, многократная конкатенация в цикле создаёт много промежуточных объектов и снижает производительность. Для изменяемых

строк используются `StringBuilder` (несинхронизированный, быстрее) и `StringBuffer` (синхронизированный, потокобезопасный, медленнее).

Проблема с обычной конкатенацией

```
public class ConcatPerformance {
    public static void main(String[] args) {
        // ПЛОХО: создаётся много промежуточных строк
        String result = "";
        for (int i = 0; i < 100; i++) {
            result += i; // каждая итерация создаёт новую строку
        }

        // ХОРОШО: используется StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 100; i++) {
            sb.append(i);
        }
        String result2 = sb.toString();
    }
}
```

Основные методы `StringBuilder`

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        // Создание
        StringBuilder sb = new StringBuilder();           // пустой
        StringBuilder sb2 = new StringBuilder("Hello");  // со строкой
        StringBuilder sb3 = new StringBuilder(50);        // с начальной ёмкостью

        // append() — добавление
        sb.append("Java");
        sb.append(" ");
        sb.append("is");
        sb.append(" ");
        sb.append("awesome");
        System.out.println(sb); // "Java is awesome"

        // Цепочка вызовов (method chaining)
        sb = new StringBuilder();
        sb.append("Hello").append(" ").append("World");
        System.out.println(sb); // "Hello World"
    }
}
```

```

// insert() — вставка по индексу
sb.insert(6, "Beautiful ");
System.out.println(sb); // "Hello Beautiful World"

// delete() — удаление
sb.delete(6, 16);
System.out.println(sb); // "Hello World"

// replace() — замена
sb.replace(0, 5, "Hi");
System.out.println(sb); // "Hi World"

// reverse() — переворот
sb.reverse();
System.out.println(sb); // "dlroW iH"

// length() и capacity()
System.out.println("Длина: " + sb.length()); // 8
System.out.println("Ёмкость: " + sb.capacity()); // зависит от реализации

// toString() — преобразование в String
String finalString = sb.toString();
}
}

```

Когда использовать String, а когда StringBuilder

Ситуация	Рекомендация
Небольшое количество конкатенаций (до 5-10)	String (компилятор сам оптимизирует)
Много конкатенаций в цикле	StringBuilder
Конкатенация в однопоточной среде	StringBuilder
Конкатенация в многопоточной среде	StringBuffer
Строка не меняется после создания	String

Таблица 1. Использование String

Итоги шестой лекции

Сегодня мы изучили:

Класс String:

- Строки неизменяемы (immutable)
- Сравнение строк: всегда equals(), никогда ==
- Основные методы: length(), charAt(), indexOf(), substring(), split()
- Преобразование регистра: toLowerCase(), toUpperCase()
- Замена: replace(), replaceAll()
- Преобразование типов: valueOf(), parseInt() и т.д.

Классы StringBuilder и StringBuffer:

- Для изменяемых строк
- append(), insert(), delete(), replace(), reverse()
- StringBuilder быстрее, StringBuffer — потокобезопасен

Ключевые выводы:

1. Строки в Java — это объекты, а не массивы символов
2. Неизменяемость строк даёт преимущества в безопасности и производительности при работе с пулом строк
3. Для многократной конкатенации в цикле используйте StringBuilder
4. Всегда помните про NullPointerException при работе со строками